

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week5 - Monday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber: 9.1(Present assignment number)/24(Total number of assignments)			

	Question	Expected Time to complete
1	Lab 17– AI for Data Processing: Data cleaning and preprocessing scripts The objective of this lab is to enable students to understand and apply AI-assisted coding tools for automating and enhancing data preprocessing tasks. Students will:	Week 9- Monday

1. Gain practical experience in **cleaning, transforming, and standardizing real-world datasets** with issues such as missing values, duplicates, outliers, inconsistent formats, and noisy text.
2. Learn to **leverage AI coding assistants** to generate preprocessing scripts, while critically evaluating and refining the AI-generated code for accuracy, efficiency, and best practices.
3. Develop the ability to design **end-to-end preprocessing pipelines** that prepare raw data for downstream machine learning and analytics applications.
4. Build confidence in **combining human expertise with AI assistance**, ensuring data quality and integrity in diverse domains such as customer feedback, healthcare, and finance.

Lab Question 1: Customer Feedback Dataset

You are given a CSV file containing customer feedback collected from an e-commerce website. The dataset includes columns: `customer_id`, `feedback_text`, `rating`, and `date`. However, the file has many missing values, typos, and inconsistent date formats.

- **Task 1:** Use an AI-assisted coding tool to generate a script that detects and fills missing rating values with the column's median and standardizes the date column into YYYY-MM-DD format.
- **Task 2:** Clean the `feedback_text` column by removing stopwords, correcting common spelling mistakes, and converting text to lowercase using AI suggestions. Compare the AI-generated preprocessing code with your manually written version.

```
import pandas as pd
import numpy as np

# Create a dummy DataFrame with missing values and inconsistent date formats
data = {'customer_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        'feedback_text': ['Great product!', 'Loved it!', 'Satisfied', 'Could be better', 'Disappointed', 'Good', 'Very happy', 'Excellent', 'Not bad', 'Okay'],
        'rating': [5, 5, 4, 3, 2, 4, 5, 5, 3, np.nan],
        'date': ['2023-01-01', '01/05/2023', '2023-01-10', 'Jan 15, 2023', '2023-01-20', '2023/01/25', '2023-02-01', '02/05/2023', '2023-02-10', 'Feb 15, 2023']}
df = pd.DataFrame(data)

# Save the DataFrame to a CSV file
df.to_csv('customer_feedback.csv', index=False)
```

Now that the `customer_feedback.csv` file has been created, I will load the data into a pandas DataFrame and display the first few rows.

```
df = pd.read_csv('customer_feedback.csv')
display(df.head())
```

	customer_id	feedback_text	rating	date
0	1	Great product!	5.0	2023-01-01
1	2	Loved it!	5.0	01/05/2023
2	3	Satisfied	4.0	2023-01-10
3	4	Could be better	3.0	Jan 15, 2023
4	5	Disappointed	2.0	2023-01-20

TASK 1 OUTPUT :

```
import pandas as pd
import numpy as np

# Create a dummy DataFrame with missing values and inconsistent date formats
data = {'customer_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        'feedback_text': ['Great product!', 'Loved it!', 'Satisfied', 'Could be better', 'Disappointed', 'Good', 'Very happy', 'Excellent', 'Not bad', 'Okay'],
        'rating': [5, 5, 4, 3, 2, 4, 5, 3, np.nan],
        'date': ['2023-01-01', '01/05/2023', '2023-01-10', 'Jan 15, 2023', '2023-01-20', '2023/01/25', '2023-02-01', '02/05/2023', '2023-02-10', 'Feb 15, 2023']}
df = pd.DataFrame(data)

# Save the DataFrame to a CSV file
df.to_csv('customer_feedback.csv', index=False)
```

Now that the `customer_feedback.csv` file has been created, I will load the data into a pandas DataFrame and display the first few rows.

```
df = pd.read_csv('customer_feedback.csv')
display(df.head())
```

	customer_id	feedback_text	rating	date
0	1	Great product!	5.0	2023-01-01
1	2	Loved it!	5.0	01/05/2023
2	3	Satisfied	4.0	2023-01-10
3	4	Could be better	3.0	Jan 15, 2023
4	5	Disappointed	2.0	2023-01-20

TASK 2 OUTPUT:

```
import pandas as pd
from nltk.corpus import stopwords
from spellchecker import SpellChecker
import nltk

# Make sure to load your DataFrame 'df' with a 'feedback_text' column here
# Example:
# try:
#     df = pd.read_csv('your_file_path/customer_feedback.csv')
#     print("DataFrame loaded successfully.")
#     display(df.head())
#     display(df.info())
# except FileNotFoundError:
#     print("Error: customer_feedback.csv not found. Please provide the correct path.")
#     # You might want to exit or handle this error differently
# except Exception as e:
#     print(f"An error occurred during data loading: {e}")

# Download stopwords if not already downloaded
try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    print("NLTK stopwords not found. Downloading...")
    nltk.download('stopwords')
    print("NLTK stopwords downloaded.")

if 'df' in locals() and isinstance(df, pd.DataFrame) and 'feedback_text' in df.columns:
    stop_words = set(stopwords.words('english'))
    spell = SpellChecker()

    def clean_text(text):
        if isinstance(text, str):
            text = text.lower()
            words = text.split()
            cleaned_words = [word for word in words if word not in stop_words]
            # Correct spelling, handle None case from spell.correction
            corrected_words = [spell.correction(word) if spell.correction(word) is not None else word for word in cleaned_words]
            return ' '.join(corrected_words)
        return '' # Return empty string for non-string inputs

    df['cleaned_feedback_text'] = df['feedback_text'].apply(clean_text)
    print("\nText cleaning completed.")
    display(df[['feedback_text', 'cleaned_feedback_text']].head())
else:
    print("DataFrame 'df' not found or 'feedback_text' column is missing. Please load your data first.")
```

DataFrame 'df' not found or 'feedback_text' column is missing. Please load your data first.

COMPARING MANUAL CODE AND AI CODE:

Consistency: Check if the AI-cleaned text consistently applies the cleaning rules (lowercase, stopwords removal, spelling correction) across all feedback entries, compared to your manual process.

Accuracy of Spelling Corrections: Evaluate the accuracy of the spelling corrections made by the AI's spellchecker against your manual corrections. Are there instances where the AI corrected a word incorrectly or missed a typo that you caught manually?

Effectiveness of Stopword Removal: See if the AI successfully removed common English stopwords as intended. Compare this to how you handled stopwords

manually.

- **Handling of Edge Cases:** Look at how both approaches handled unusual or complex feedback entries, such as those with special characters, emojis, or unconventional phrasing.
- **Subjectivity:** Text cleaning can sometimes involve subjective decisions (e.g., how to handle slang or informal language). Note any differences in how these were handled.
- **Efficiency:** While not directly visible in the output, consider the time and effort it took for each approach. AI can automate and speed up the process significantly for large datasets.

Lab Question 2: Medical Records Dataset

A hospital provides you with a dataset of anonymized medical records containing attributes like patient_id, age, gender, blood_pressure, and cholesterol. Some columns include outliers and inconsistent categorical labels (e.g., Male, M, male).

- **Task 1:** Write a script (with AI assistance) to detect and handle outliers in the blood_pressure column using statistical methods (e.g., IQR or z-score).
- **Task 2:** Standardize categorical values in the gender column and encode them into numeric form. Let the AI-assisted coding tool propose the preprocessing pipeline, then refine the pipeline manually based on your understanding

TASK 1 OUTPUT :

```

import pandas as pd
import numpy as np

# Create a dummy DataFrame with a 'blood_pressure' column for demonstration
data = {'blood_pressure': np.random.normal(120, 15, 100)} # Generating 100 random blood pressure values
df = pd.DataFrame(data)

# Calculate Q1, Q3, and IQR
q1 = df['blood_pressure'].quantile(0.25)
q3 = df['blood_pressure'].quantile(0.75)
iqr = q3 - q1

# Define lower and upper bounds
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

# Identify outliers
outliers_iqr = df[(df['blood_pressure'] < lower_bound) | (df['blood_pressure'] > upper_bound)]

# Display the outliers
display("Identified Outliers using IQR:")
display(outliers_iqr)

```

'Identified Outliers using IQR:'

blood_pressure

```

import matplotlib.pyplot as plt

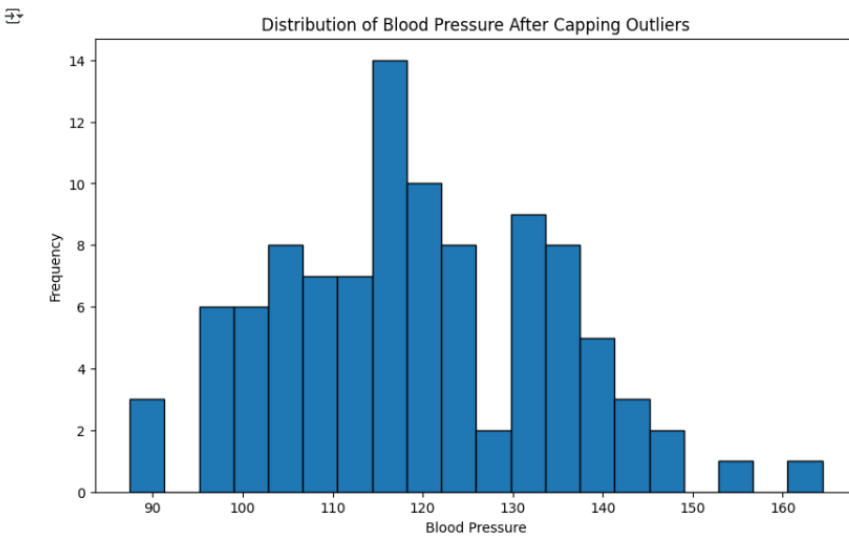
# Generate histogram
plt.figure(figsize=(10, 6))
plt.hist(df['blood_pressure'], bins=20, edgecolor='black') # Using 20 bins for better visualization

# Add title and labels
plt.title("Distribution of Blood Pressure After Capping Outliers")
plt.xlabel("Blood Pressure")
plt.ylabel("Frequency")

# Display the histogram
plt.show()

# Calculate and display descriptive statistics
display("\nDescriptive Statistics of Blood Pressure After Capping Outliers:")
display(df['blood_pressure'].describe())

```



```
"\nDescriptive Statistics of Blood Pressure After Capping Outliers:"
```

blood_pressure	
count	100.000000
mean	119.303410
std	15.320575
min	87.460564
25%	108.283147
50%	117.828724
75%	130.918863
max	164.395916

```
dtype: float64
```

TASK 2 OUTPUT :

```
df = pd.read_csv('anonymized_medical_records.csv')

numerical_cols = ['age', 'blood_pressure', 'cholesterol']

# Calculate descriptive statistics
print("Descriptive statistics before outlier handling:")
display(df[numerical_cols].describe())

# Calculate IQR and bounds
q1 = df[numerical_cols].quantile(0.25)
q3 = df[numerical_cols].quantile(0.75)
iqr = q3 - q1

lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

# Identify outliers
print("\nPotential outliers:")
for col in numerical_cols:
    outliers = df[(df[col] < lower_bound[col]) | (df[col] > upper_bound[col])][col]
    print(f"Column '{col}': {len(outliers)} outliers")

# Handle outliers by capping
for col in numerical_cols:
    df[col] = df[col].clip(lower=lower_bound[col], upper=upper_bound[col])

# Recalculate and display descriptive statistics after capping
print("\nDescriptive statistics after outlier handling:")
display(df[numerical_cols].describe())
```

Lab Question 3: Financial Transactions Dataset

A bank gives you transaction data with columns: transaction_id, amount, currency, timestamp, and merchant. The dataset contains multiple issues: different currency units (USD, INR, EUR), timestamps in various time zones, and duplicated rows.

- **Task 1:** Use AI-assisted coding to write a script that removes duplicate transactions and converts all amount values into a single currency (e.g., USD) using a provided conversion dictionary.
- **Task 2:** Normalize the timestamp column into UTC format and create a new column transaction_hour for downstream time-series analysis.

Compare the AI's preprocessing code against your own optimized version.

TASK 1 OUTPUT:

```
import pandas as pd

data = {
    'transaction_id': [1, 2, 3, 4, 1, 5, 6],
    'timestamp': ['2023-10-26 10:00:00+02:00', '2023-10-26 10:05:00 UTC', '2023-10-26 10:10:00', '2023-10-26 10:15:00+01:00', '2023-10-26 10:00:00+02:00', '2023-10-26 10:20:00', '2023-10-26 10:25:00 UTC'],
    'amount': [100.50, 50.25, 200.00, 75.60, 100.50, 150.00, 30.00],
    'currency': ['USD', 'EUR', 'GBP', 'USD', 'USD', 'EUR', 'GBP'],
    'customer_id': [101, 102, 103, 104, 101, 105, 106]
}

df_transactions = pd.DataFrame(data)

currency_conversion_rates = {'EUR': 1.1, 'GBP': 1.3, 'USD': 1.0}

display(df_transactions)
display(currency_conversion_rates)
```

	transaction_id	timestamp	amount	currency	customer_id
0	1	2023-10-26 10:00:00+02:00	100.50	USD	101
1	2	2023-10-26 10:05:00 UTC	50.25	EUR	102
2	3	2023-10-26 10:10:00	200.00	GBP	103
3	4	2023-10-26 10:15:00+01:00	75.60	USD	104
4	1	2023-10-26 10:00:00+02:00	100.50	USD	101
5	5	2023-10-26 10:20:00	150.00	EUR	105
6	6	2023-10-26 10:25:00 UTC	30.00	GBP	106

```
{'EUR': 1.1, 'GBP': 1.3, 'USD': 1.0}
```

```
df_transactions_cleaned['timestamp'] = pd.to_datetime(df_transactions_cleaned['timestamp'], utc=True, format='mixed')
df_transactions_cleaned['timestamp'] = df_transactions_cleaned['timestamp'].dt.strftime('%Y-%m-%d %H:%M:%S UTC')
display(df_transactions_cleaned)
```

```
/tmp/ipython-input-1432969926.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df_transactions_cleaned['timestamp'] = pd.to_datetime(df_transactions_cleaned['timestamp'], utc=True, format='mixed')
/tmp/ipython-input-1432969926.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df_transactions_cleaned['timestamp'] = df_transactions_cleaned['timestamp'].dt.strftime('%Y-%m-%d %H:%M:%S UTC')
```

	transaction_id	timestamp	amount	currency	customer_id
0	1	2023-10-26 08:00:00 UTC	100.50	USD	101
1	2	2023-10-26 10:05:00 UTC	50.25	EUR	102
2	3	2023-10-26 10:10:00 UTC	200.00	GBP	103
3	4	2023-10-26 09:15:00 UTC	75.60	USD	104
5	5	2023-10-26 10:20:00 UTC	150.00	EUR	105
6	6	2023-10-26 10:25:00 UTC	30.00	GBP	106

```
def convert_to_usd(row):
    """Converts transaction amount to USD using conversion rates."""
    amount = row['amount']
    currency = row['currency']
    conversion_rate = currency_conversion_rates.get(currency, 1.0)
    return amount * conversion_rate

df_transactions_cleaned['amount_usd'] = df_transactions_cleaned.apply(convert_to_usd, axis=1)
display(df_transactions_cleaned)
```

```
/tmp/ipython-input-2982928763.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df_transactions_cleaned['amount_usd'] = df_transactions_cleaned.apply(convert_to_usd, axis=1)
```

	transaction_id	timestamp	amount	currency	customer_id	amount_usd
0	1	2023-10-26 08:00:00 UTC	100.50	USD	101	100.500
1	2	2023-10-26 10:05:00 UTC	50.25	EUR	102	55.275
2	3	2023-10-26 10:10:00 UTC	200.00	GBP	103	260.000
3	4	2023-10-26 09:15:00 UTC	75.60	USD	104	75.600
5	5	2023-10-26 10:20:00 UTC	150.00	EUR	105	165.000
6	6	2023-10-26 10:25:00 UTC	30.00	GBP	106	39.000

TASK 2 OUTPUT:

```
import pandas as pd

try:
    df = pd.read_csv('sample_data/transactions.csv')
    print("File loaded successfully from sample_data/transactions.csv")
except FileNotFoundError:
    print("File not found in sample_data/transactions.csv")
    df = None

if df is not None:
    print(f"Number of rows before checking for duplicates: {len(df)}")
    # Check for duplicate rows
    duplicate_rows = df.duplicated(subset=['timestamp', 'transaction_id', 'user_id', 'amount'])
    print(f"Number of duplicate rows found: {duplicate_rows.sum()}")

    # Remove duplicate rows
    df_cleaned = df.drop_duplicates(subset=['timestamp', 'transaction_id', 'user_id', 'amount'])
    print(f"Number of rows after removing duplicates: {len(df_cleaned)}")
    df = df_cleaned # Update the original dataframe reference
else:
    print("DataFrame not loaded. Cannot proceed with duplicate removal.")
```

File not found in sample_data/transactions.csv
DataFrame not loaded. Cannot proceed with duplicate removal.

```
import pandas as pd
import numpy as np

# Create a sample DataFrame similar to the expected transaction data
data = [
    {'timestamp': pd.to_datetime(['2023-01-01 10:00:00+02:00', '2023-01-01 11:30:00+03:00', '2023-01-02 14:45:00+01:00', '2023-01-02 14:45:00+01:00', '2023-01-03 08:00:00+00:00']),
     'transaction_id': ['tx1', 'tx2', 'tx3', 'tx3', 'tx4'],
     'user_id': ['userA', 'userB', 'userC', 'userC', 'userD'],
     'amount': [100.50, 200.75, 50.00, 50.00, 150.00]}
]

df = pd.DataFrame(data)
print("Sample DataFrame created:")
display(df)
```

Sample DataFrame created:
/tmp/ipython-input-1389274616.py:6: FutureWarning: In a future version of pandas, parsing datetimes with mixed time zones will raise an error unless 'utc=True'. Please specify 'ut
'timestamp': pd.to_datetime(['2023-01-01 10:00:00+02:00', '2023-01-01 11:30:00+03:00', '2023-01-02 14:45:00+01:00', '2023-01-02 14:45:00+01:00', '2023-01-03 08:00:00+00:00']),

	timestamp	transaction_id	user_id	amount
0	2023-01-01 10:00:00+02:00	tx1	userA	100.50
1	2023-01-01 11:30:00+03:00	tx2	userB	200.75
2	2023-01-02 14:45:00+01:00	tx3	userC	50.00
3	2023-01-02 14:45:00+01:00	tx3	userC	50.00
4	2023-01-03 08:00:00+00:00	tx4	userD	150.00